

VERIFIABILITY, ERROR LOCALIZATION, AND LOCAL REPAIRABILITY

UNIQUE SUBEDI

Whenever we ask why AI is so remarkable at tasks such as writing code or solving difficult mathematical problems, one common explanation is the verifiability of these domains. Programs can be compiled and tested, and mathematical arguments can be checked step by step or formalized and verified using proof assistants. Thus, a sufficiently large number of AI agents can keep proposing solutions, verifying them, and eventually arrive at a correct answer.

However, I do not think verifiability alone explains why this process is feasible. To see this, note that for any problem whose solutions admit a finite encoding, finding a solution can be represented as identifying a member of a nonempty set

$$\mathcal{S}_X \subseteq \{0, 1\}^T,$$

where $\mathcal{S}_X$ contains the binary strings representing valid solutions to the problem X . These may include multiple equivalent math proofs or programs that pass unit tests.

Suppose we can easily verify whether a proposed string belongs to $\mathcal{S}_X$. There are still 2^T possibilities. If $M = |\mathcal{S}_X|$, independent uniform sampling requires $2^T/M$ verification rounds on average. In the worst case, there may be only one valid string, so blind search still requires $\Omega(2^T)$ rounds. Here, I will measure complexity only by the number of iterations, setting aside the cost of each verification.

An easy counterargument is that an AI agent is not searching uniformly over this enormous space. Real solutions have structure, and a trained model may assign much more probability to valid solutions than uniform sampling would. If $q(y \mid X)$ is the model's distribution over candidates, its probability of producing a valid solution is

$$p_X := \sum_{y \in \mathcal{S}_X} q(y \mid X).$$

Provided $p_X > 0$, independent sampling requires $1/p_X$ attempts on average. A learned prior can therefore substantially reduce the search effort, but verifiability alone gives no guarantee that p_X is large. Since repeated single-shot generation of solution from the problem description alone rarely succeed, p_X is most likely very small in practice.

For example, suppose that at $h(T)$ specified bit positions, conditional on the preceding prefix still admitting a valid completion, the next bit rules out every valid completion with probability at least a fixed $\varepsilon \in (0, 1)$. Then

$$p_X \leq (1 - \varepsilon)^{h(T)} \leq e^{-\varepsilon h(T)}.$$

Independent sampling therefore requires at least $e^{\varepsilon h(T)}$ attempts on average. If $h(T) = T^\alpha$ for any fixed $\alpha \in (0, 1)$, this grows faster than any polynomial in T , even though $h(T) \ll T$.

More importantly, this sampling picture misses an important part of how AI agents make progress. They can use feedback from failed attempts to improve an existing candidate. Accordingly, in addition to verifiability, I think two properties help explain success in these domains: *error localization* and *local repairability*.

The localization of errors matters because a binary verifier only tells us whether a candidate is correct. More informative feedback can help us identify where it is wrong. A compiler may flag a particular error, a unit test may narrow down a failing behavior, or a proof checker may point to an invalid step. Local repairability means that, once we have located an error, we can fix it while

preserving most of what is already correct. If every correction requires starting over, localization alone does not get us very far.

The process is therefore not simply

$$\text{generate} \rightarrow \text{verify} \rightarrow \text{repeat}.$$

Instead, it is

$$\text{generate} \rightarrow \text{verify} \rightarrow \text{localize the error} \rightarrow \text{repair locally} \rightarrow \text{repeat}.$$

How many rounds might this take? Let

$$D_k = \min_{Y^* \in \mathcal{S}_X} |\{i : Y_i^{(k)} \neq Y_i^*\}|$$

be the minimum number of bit changes needed to turn the candidate after round k into a valid solution. Thus, $D_k = 0$ exactly when $Y^{(k)} \in \mathcal{S}_X$. The closest valid solution may change from one round to the next. Initially, the distance can be as large as T .

For illustration, suppose each round identifies and fixes at least a constant number m of defects relative to a closest valid solution, while preserving the components that already agree with that solution. If fewer than m defects remain, suppose it fixes those remaining defects. Then

$$D_{k+1} \leq \max\{D_k - m, 0\}.$$

We therefore need at most $\lceil T/m \rceil$ rounds, which is $O(T)$ for constant m .

However, repairs need not improve the candidate in every round. To allow such setbacks, we can assume

$$D_{k+1} = \min\{T, \max\{0, D_k - m + \eta_k\}\},$$

stopping once $D_k = 0$. Here, $\eta_k > 0$ represents the random regressions from the intended repair. Suppose these are independent, nonnegative integer-valued random variables with a common mean,

$$\mathbb{E}[\eta_k] = \mu < m.$$

For fixed m and a gap $m - \mu > 0$, the expected number of rounds is still $O(T)$. The same reasoning allows less frequent progress: if each net reduction of one defect takes at most n rounds on average, the expected bound becomes $O(nT)$. Thus, incomplete feedback and occasional new errors does not prevent efficient repair, provided the average repair gain exceeds the average setback.

In fact, the improvement can be even faster. Suppose each round removes a constant fraction of the remaining defects:

$$D_{k+1} \leq \alpha D_k, \quad 0 < \alpha < 1.$$

Then $D_k \leq \alpha^k D_0$. This implies that only $O(\log T)$ rounds are needed, which seem closer to what we observe in practice.

Therefore, together with verifiability, error localization and local repairability may help explain AI's effectiveness in these domains.